Annexe : UTILISATION DE LA BIBLIOTHEQUE DE SEMAPHORES LIBSEM

La bibliothèque de sémaphores, appelée `libsem`, est un outil facilitant l'utilisation des sémaphores au-dessus d'Unix. Elle reprend l'interface classique des sémaphores définie par E. Dijkstra

1. LES PRIMITIVES

Les sémaphores sont de type *dijkstra_sem_t* et la mémoire partagée de type *dijkstra_shm_t* définis dans le fichier *semaphore_dijkstra.h*

libsem contient les primitives suivantes qui retournent 0 en cas de succès et un numéro d'erreur (errno) différent de 0 en cas d'échec:

```
#include "semaphore_dijkstra.h"

int sem_init_dijkstra(dijkstra_sem_t *s, unsigned int val);
```

Fonction qui crée le sémaphore *s* initialisé à la valeur *val*

```
int P(dijkstra_sem_t *s);
```

Réalisation de la primitive P sur le sémaphore *s*.

```
int V(dijkstra_sem_t *s);
```

Réalisation de la primitive V sur le sémaphore *s*.

```
int sem_destroy_dijkstra(dijkstra_sem_t *s);
```

Destruction du sémaphore *s*.

```
int shm_init_dijkstra(dijkstra_shm_t *shm, size_t size);
```

Création du segment de mémoire partagée *s* de taille *size*.

```
int shm_destroy_dijkstra(dijkstra_shm_t *shm);
```

Destruction du segment de mémoire partagée *s*.

4. POUR INSTALLER LA LIBRAIRIE CHEZ SOI

La librairie est disponible sous forme d'archive compressée sur le moodle de l'UE à l'url :
https://moodle-sciences-26.sorbonneuniversite.fr/pluginfile.php/59059/mod_folder/content/0/libsem.tgz?forcedownload=1

Pour la décompresser il suffit de taper sur votre machine :

```
$ tar xvfz libsem.tgz
```

Un répertoire libsem est créé contenant les trois répertoires `src`, `lib`, `demo`, `tmeAddMult`. Il suffit de compiler la librairie :

```
$ cd libsem/src
$ make
```

Vous pouvez alors compiler l'exemple présenté en section 5 et l'exécuter :

```
$ cd ../demo
$ make
$ ./demo
```

Vous trouverez aussi les fichiers pour le TME dans le repertoire tmeAddMult.

5. EXEMPLE D'UTILISATION

Le programme ci-dessous réalise une barrière à trois processus. Le processus père crée deux fils et les attend à la barrière.

```
/* main.c - Exemple d'utilisation du sémaphore et mémoire partagée */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore_dijkstra.h>

#define NCHILD 4

/* définition de la mémoire partagée */
typedef struct {
    int counter;
    char flag;
} shared_data_t;

static dijkstra_shm_t shm;
static shared_data_t *shared;

/* Définition des sémaphores */

static dijkstra_sem_t semB[NCHILD]; /* Pour bloquer les fils */
static dijkstra_sem_t mutex; /* Pour l'accès à la mémoire partagé en SC */


/* Code des processus fils */

void child(int id) {
  printf("Début processus %d\n", id);

  /* Attendre que le procesus main reveille */
  P(&semB[id]);

  /* Accéder aux variables partagées en mutex */
  P(&mutex);
  shared->counter++;
  shared->flag += id;
  V(&mutex);

  printf("Fin processus %d\n", id);
  exit(0);
}

/* Destruction des sémaphores */

void destroy_sem() {
  for (int i  = 0; i < NCHILD; i++)
    sem_destroy_dijkstra(&semB[i]);
  sem_destroy_dijkstra(&mutex);
}
```

```
int main(void) {

  /* Initialisation des sémaphores */

  for (int i=0; i < NCHILD; i++) {
    if (sem_init_dijkstra(&semB[i], 0)) {
      perror("sem_init_dijkstra");
      return 1;
    }
  }

  if (sem_init_dijkstra(&mutex, 1)) {
    perror("sem_init_dijkstra");
    return 1;
  }

  /* Création de la mémoire partagée */

  if (shm_init_dijkstra(&shm, sizeof(shared_data_t))) {
    perror("shm_init_dijkstra");
    destroy_sem();
    return 1;
  }

  shared = (shared_data_t*)shm.ptr;

  /* Initialisation de la mémoire partagée */

  shared->counter = 0;
  shared->flag = 'A';

  /* Création des processus fils */
  for (int i = 0; i < NCHILD; ++i) {
    pid_t pid = fork();
    if (pid < 0) {
      perror("fork");
      exit(1);
    }
    if (pid == 0) {
      child(i);
    }
  }


  /* Reveiller les fils */

  for (int i = 0; i < NCHILD; ++i)
    V(&semB[i]);

  /* Attendre tous les fils */
  for (int i = 0; i < NCHILD; ++i)
    wait(NULL);

  printf("Tous les processus  terminés, counter final = %d, flag final = %c\n",
shared->counter, shared->flag);

  /* Destruction */
  shm_destroy_dijkstra(&shm);
  destroy_sem();

  return 0;
}
```